

Computer Science and Engineering Department, SVNIT, Surat

M.Tech. I – Semester I

Machine Learning (CSDS105)

Lab Assignment 4: Unsupervised Learning – K-Means and Hierarchical Clustering

1 Introduction and Theoretical Foundations

Unsupervised learning addresses the problem of inferring hidden structure from unlabelled multivariate observations. Clustering partitions an unlabelled dataset $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$ into K disjoint subsets (clusters) $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$ such that intra-cluster instances maximize pairwise similarity while inter-cluster instances maximize dissimilarity.

In this laboratory study, we conduct an experimental analysis of two core clustering paradigms applied to Fisher's benchmark Iris flower dataset:

1. **Centroid-Based Partitioning (K-Means Clustering):** An iterative optimization algorithm that partitions the feature space into Voronoi cells parameterized by cluster centroids.
2. **Agglomerative Hierarchical Clustering:** A connectivity-based bottom-up algorithm producing an interpretable nested dendrogram hierarchy using distance metrics and linkage criteria.

1.1 K-Means Clustering Formulation

Given a pre-specified cluster count K , K-Means minimizes the Within-Cluster Sum of Squares (WCSS), formally termed the clustering inertia:

$$J(\mathcal{C}, \boldsymbol{\mu}) = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2 \quad (1)$$

where $\boldsymbol{\mu}_k \in \mathbb{R}^d$ represents the geometric centroid of cluster C_k , computed as:

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i \quad (2)$$

Lloyd's algorithm achieves local convergence through alternating optimization:

1. **Assignment Step:** Each instance $\mathbf{x}_i$ is assigned to its nearest centroid:

$$c^{(i)} = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2 \quad (3)$$

2. **Update Step:** Centroid coordinates are recomputed as the empirical mean of all members assigned to that partition:

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{|C_k^{(t)}|} \sum_{\mathbf{x}_i \in C_k^{(t)}} \mathbf{x}_i \quad (4)$$

Convergence is reached when assignments cease to mutate or when centroid displacement $\|\boldsymbol{\mu}_k^{(t+1)} - \boldsymbol{\mu}_k^{(t)}\|$ falls below numerical tolerance ϵ .

1.2 Hierarchical Agglomerative Clustering Formulation

Hierarchical clustering builds a bottom-up dendrogram without requiring a pre-specified number of clusters at initialization. Each observation begins in its own singleton cluster $C_i = \{\mathbf{x}_i\}$. At each subsequent step, the pair of clusters exhibiting the minimum inter-cluster dissimilarity $d(A, B)$ is merged:

$$(A^*, B^*) = \arg \min_{A, B} d(A, B) \quad (5)$$

The mathematical choice of linkage metric $d(A, B)$ governs the geometric topology of the resulting clusters:

- **Ward's Minimum Variance Linkage:** Merges clusters that minimize the total within-cluster variance increase (Error Sum of Squares, ESS):

$$\Delta \text{ESS}(A, B) = \frac{|A||B|}{|A| + |B|} \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|_2^2 \quad (6)$$

- **Complete Linkage (Maximum Dissimilarity):** Enforces compact, spherical boundaries by evaluating the maximum pairwise sample distance:

$$d_{\text{complete}}(A, B) = \max_{\mathbf{x} \in A, \mathbf{y} \in B} \|\mathbf{x} - \mathbf{y}\|_2 \quad (7)$$

- **Average Linkage (UPGMA):** Computes the mean pairwise Euclidean distance between all cross-cluster sample pairs:

$$d_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{\mathbf{x} \in A} \sum_{\mathbf{y} \in B} \|\mathbf{x} - \mathbf{y}\|_2 \quad (8)$$

1.3 Cluster Validation Metrics

Because true class labels are typically inaccessible in unsupervised learning, quality must be evaluated using intrinsic geometric properties:

1. **Silhouette Coefficient:** Quantifies how well-separated each sample is from neighboring clusters. For sample i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (9)$$

where $a(i) = \frac{1}{|C_I|-1} \sum_{j \in C_I, j \neq i} \|\mathbf{x}_i - \mathbf{x}_j\|$ denotes mean intra-cluster distance, and $b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} \|\mathbf{x}_i - \mathbf{x}_j\|$ denotes minimum mean nearest-cluster distance. The global score is $S = \frac{1}{N} \sum_{i=1}^N s(i) \in [-1, 1]$.

2. **Davies-Bouldin Index (DBI):** Evaluates the maximum ratio of intra-cluster dispersion to inter-centroid separation:

$$DB = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \left(\frac{\bar{d}_i + \bar{d}_j}{\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|_2} \right) \quad (10)$$

Lower DBI values denote more compact clusters with wider mutual separation.

3. **Calinski-Harabasz Index (Variance Ratio Criterion):** Ratio of the between-cluster dispersion matrix trace to the within-cluster dispersion matrix trace:

$$CH = \frac{\text{Tr}(\mathbf{B}_K)}{\text{Tr}(\mathbf{W}_K)} \cdot \frac{N - K}{K - 1} \quad (11)$$

2 Environment Setup and Exploratory Data Analysis

2.1 Experimental Setup and Preprocessing Pipeline

The computational environment is built on Python 3 using NumPy, Pandas, Matplotlib, Seaborn, SciPy, and Scikit-Learn. The Iris dataset consists of $N = 150$ observations across 4 continuous morphological features:

- Sepal Length (cm), Sepal Width (cm)
- Petal Length (cm), Petal Width (cm)

Because Euclidean distance is sensitive to variance disparities across dimensions, features are standardized via z -score normalization:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j} \quad (12)$$

Listing 1: Dataset Loading, Standardization, and Environment Initialization

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.datasets import load_iris
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.cluster import KMeans, AgglomerativeClustering
8 from sklearn.decomposition import PCA
9 from scipy.cluster.hierarchy import dendrogram, linkage
10
11 # Load Fisher's Iris Dataset
12 iris = load_iris()
13 df = pd.DataFrame(iris.data, columns=iris.feature_names)
14 df["species"] = [iris.target_names[i] for i in iris.target]
15 X, y = iris.data, iris.target
16
17 # Standard Scaling
18 scaler = StandardScaler()
19 X_scaled = scaler.fit_transform(X)
20 print("Dataset shape:", X_scaled.shape)
21 print("Class distribution:\n", df["species"].value_counts())

```

Listing 2: Dataset Summary Output

```

Dataset shape: (150, 4)
Class distribution:
  setosa      50
  versicolor  50
  virginica   50
Name: species, dtype: int64

```

2.2 Exploratory Visualizations

Univariate histograms combined with Kernel Density Estimation (KDE) and bivariate correlation analyses are depicted in Figure 1 and Figure 2.

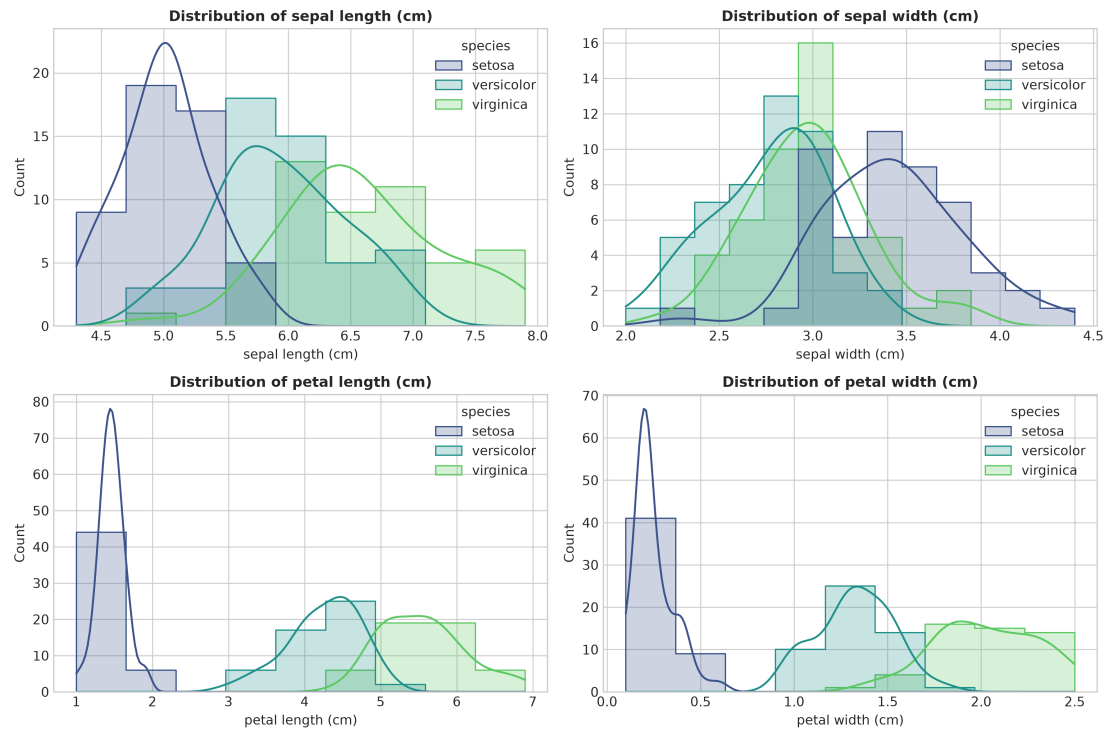


Figure 1: Empirical feature distributions (Histograms + KDE) partitioned by species across morphological dimensions.

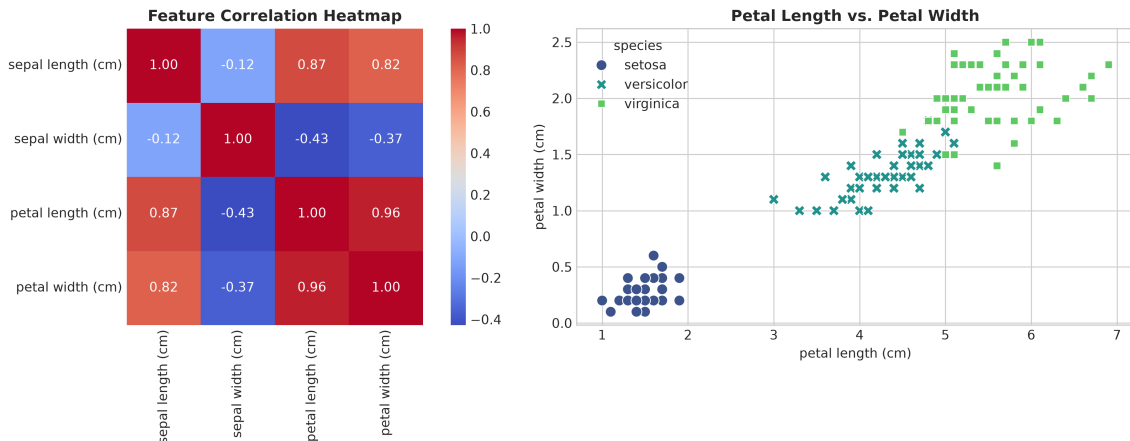


Figure 2: Left: Pearson correlation matrix of standardized features. Right: Bivariate scatter plot of Petal Length vs. Petal Width illustrating linear separability of *Setosa* and partial overlap between *Versicolor* and *Virginica*.

The exploratory analysis demonstrates that:

1. Petal length and petal width exhibit a strong positive linear correlation ($r = 0.96$).
2. *Iris setosa* forms a distinct, isolated cluster in lower petal dimensions.
3. *Iris versicolor* and *Iris virginica* form a continuous spectrum with minor overlap in feature space, presenting an authentic clustering challenge.

3 K-Means Clustering: Model Selection and Training

3.1 Hyperparameter Sweeps: Elbow Method and Silhouette Analysis

To determine the optimal number of clusters k without assuming ground-truth labels, a hyperparameter sweep over $k \in \{1, 2, \dots, 10\}$ is conducted. Inertia (WCSS) and mean Silhouette scores are computed at each candidate step.

Listing 3: Hyperparameter Sweep for Optimal k Determination

```

1 k_range = range(1, 11)
2 inertias, silhouette_scores = [], []
3
4 for k in k_range:
5     km = KMeans(n_clusters=k, random_state=42, n_init=10)
6     km.fit(X_scaled)
7     inertias.append(km.inertia_)
8     if k >= 2:
9         silhouette_scores.append(silhouette_score(X_scaled, km.labels_))
10    else:
11        silhouette_scores.append(np.nan)

```

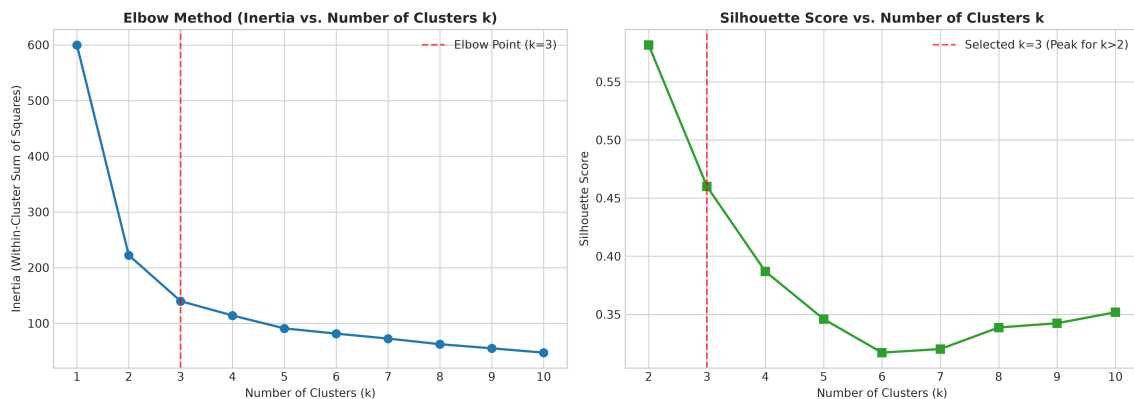


Figure 3: Model selection curves for K-Means: (Left) Elbow Method indicating inflection point at $k = 3$; (Right) Silhouette score sweep highlighting peak separation at $k = 2$ and the highest natural domain inflection at $k = 3$.

As evident from Figure 3, the rate of reduction in inertia drops sharply after $k = 3$ (the “elbow” point). While $k = 2$ achieves a marginally higher silhouette score (due to the wide separation between Setosa and the other two species combined), $k = 3$ captures the optimal balance between high silhouette quality ($S = 0.4599$) and domain-specific biological clustering.

3.2 Optimal K-Means Model Execution and Centroid Analysis

The final K-Means model is fitted with $k = 3$. The converged centroid coordinates in both standardized z -space and original dimensional scale are displayed below.

Listing 4: K-Means Model Fitting and Centroid Extraction

```

1 kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
2 kmeans_labels = kmeans.fit_predict(X_scaled)

```

```

3
4 centers_scaled = kmeans.cluster_centers_
5 centers_original = scaler.inverse_transform(centers_scaled)
6
7 print("Converged Inertia:", round(kmeans.inertia_, 4))
8 print("Standardized Centroids:\n", np.round(centers_scaled, 4))
9 print("Original Scale Centroids:\n", np.round(centers_original, 4))

```

Listing 5: K-Means Centroid Output Logs

```

Converged Inertia: 139.8205
Standardized Centroids:
[[-0.0502 -0.8834  0.3477  0.2815]
 [-1.0146  0.8533 -1.305  -1.2549]
 [ 1.136   0.0884  0.9962  1.0175]]

Original Scale Centroids (Sepal Length, Sepal Width, Petal Length, Petal Width):
[[5.8019 2.6736 4.3698 1.4132]
 [5.006  3.428  1.462  0.246 ]
 [6.7809 3.0957 5.5106 1.9723]]

```

3.3 Cluster Projections in 2D Feature Space and PCA Space

To visualize cluster geometry and centroid placements:

1. Centroids are projected into the 2D Petal Length vs. Petal Width subspace.
2. Principal Component Analysis (PCA) is applied to project the 4D standardized feature space into 2 principal components, preserving 95.81% of total variance (PC1: 72.96%, PC2: 22.85%).

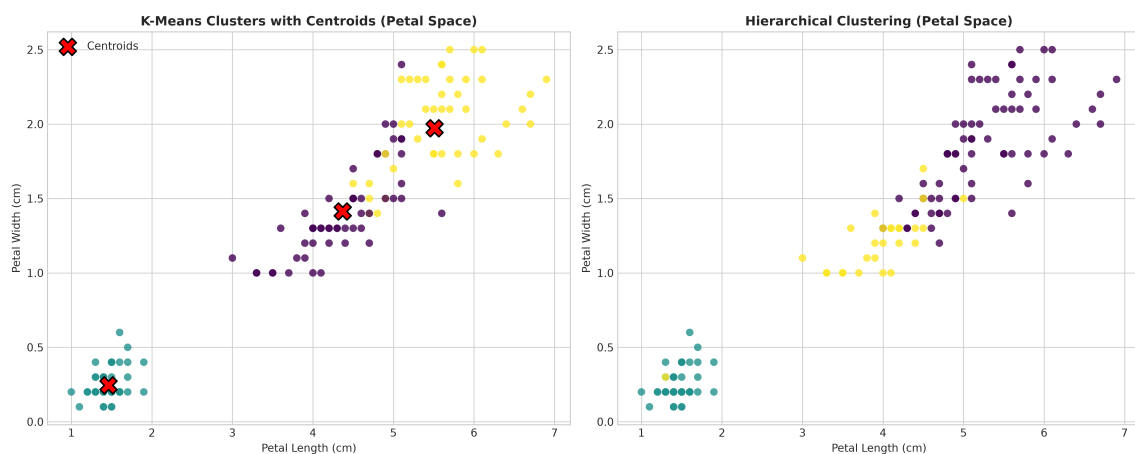


Figure 4: Cluster assignments and learned centroids projected into Petal Length vs. Petal Width feature space for K-Means (Left) and Hierarchical Clustering (Right).

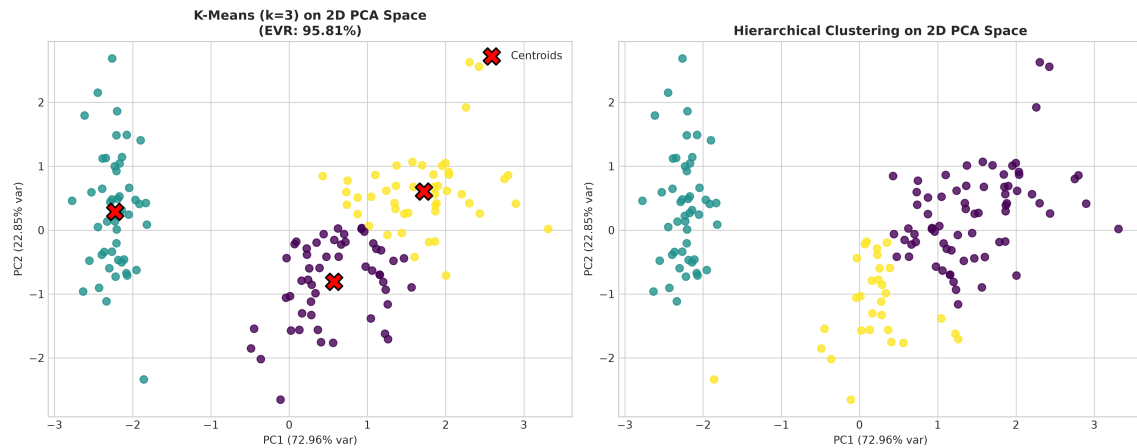


Figure 5: 2D PCA projection of clusters and transformed centroids ($k = 3$) preserving 95.81% of total variance for K-Means (Left) and Hierarchical Clustering (Right).

4 Hierarchical Agglomerative Clustering

Listing 6: Hierarchical Linkage and Dendrogram Generation

```

1 from scipy.cluster.hierarchy import dendrogram, linkage
2 from sklearn.cluster import AgglomerativeClustering
3
4 # Compute Ward Linkage Matrix
5 linkage_matrix = linkage(X_scaled, method="ward")
6
7 # Fit Agglomerative Clustering Estimator
8 hierarchical = AgglomerativeClustering(n_clusters=3, linkage="ward")
9 hier_labels = hierarchical.fit_predict(X_scaled)

```

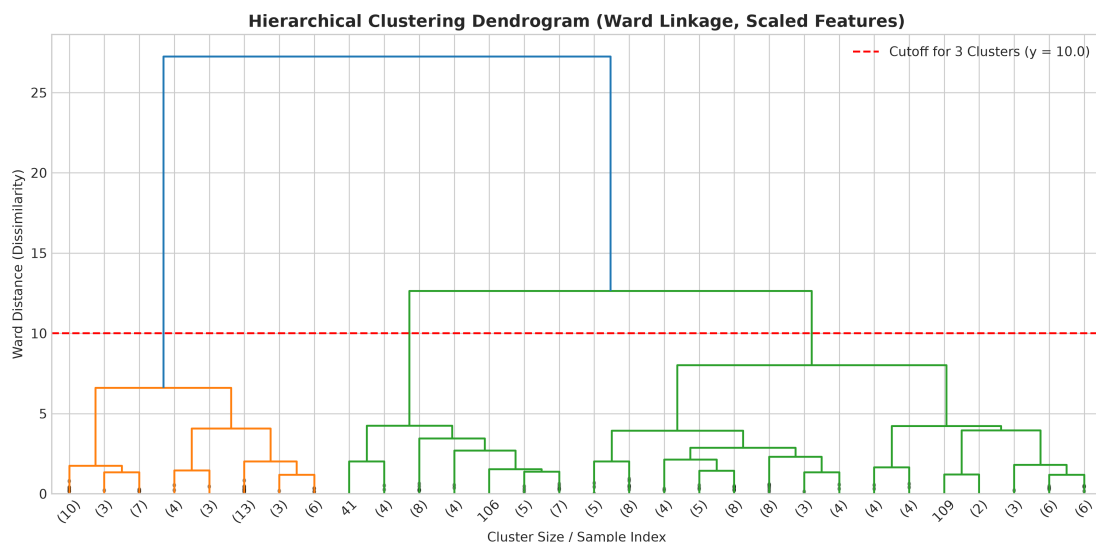


Figure 6: Hierarchical clustering dendrogram using Ward linkage on standardized features. The dashed red line at $y = 10.0$ establishes the threshold cut yielding 3 major clusters.

Cutting the dendrogram horizontally at dissimilarity threshold $y = 10.0$ yields three distinct subtrees, corroborating the structural suitability of $k = 3$.

5 Performance Benchmarking and Quantitative Validation

5.1 Evaluation Metrics and Benchmark Comparison

To validate clustering quality, both internal unsupervised geometric metrics and aligned external classification metrics against ground truth labels are computed.

Listing 7: Quantitative Performance Metric Computation

```

1 from sklearn.metrics import (
2     silhouette_score, davies_bouldin_score, calinski_harabasz_score,
3     confusion_matrix, mean_absolute_error, mean_squared_error
4 )
5
6 # Unsupervised and Supervised Benchmark Dictionary
7 metrics = {
8     "Silhouette Score": [
9         silhouette_score(X_scaled, kmeans_labels),
10        silhouette_score(X_scaled, hier_labels)
11    ],
12    "Davies-Bouldin Index": [
13        davies_bouldin_score(X_scaled, kmeans_labels),
14        davies_bouldin_score(X_scaled, hier_labels)
15    ],
16    "Calinski-Harabasz Index": [
17        calinski_harabasz_score(X_scaled, kmeans_labels),
18        calinski_harabasz_score(X_scaled, hier_labels)
19    ],
20    "MAE": [mean_absolute_error(y, kmeans_aligned), mean_absolute_error(y,
21        hier_aligned)],
22    "MSE": [mean_squared_error(y, kmeans_aligned), mean_squared_error(y, hier_aligned)
23    ],
24    "RMSE": [np.sqrt(mean_squared_error(y, kmeans_aligned)), np.sqrt(
25        mean_squared_error(y, hier_aligned))]
26 }
27
28 results_df = pd.DataFrame(metrics, index=["K-Means", "Hierarchical"])
29 print(results_df.round(4))

```

Table 1: Quantitative Performance Benchmark of Clustering Algorithms

Algorithm	Silhouette $\uparrow$	Davies-Bouldin $\downarrow$	Calinski-Harabasz $\uparrow$	MAE $\downarrow$	MSE $\downarrow$	RMSE $\downarrow$
K-Means ($k = 3$)	0.4599	0.8336	241.9044	0.1667	0.1667	0.4082
Hierarchical (Ward)	0.4467	0.8035	222.7192	0.1733	0.1733	0.4162

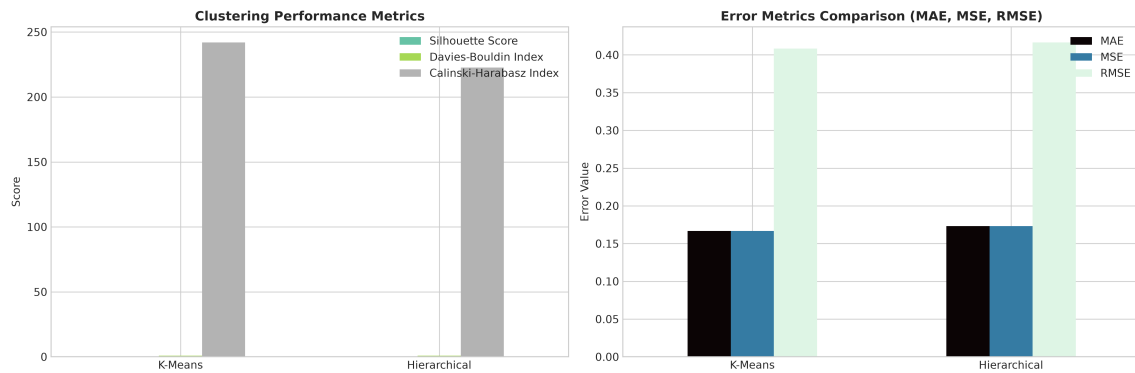


Figure 7: Comparative performance metrics: (Left) Unsupervised validation indices (Silhouette, Davies-Bouldin, Calinski-Harabasz); (Right) Alignment error metrics (MAE, MSE, RMSE).

5.2 Cluster Alignment and Confusion Matrix Analysis

Unsupervised cluster IDs are matched to ground-truth labels using majority voting alignment. The resulting confusion matrices are presented in Figure 8.

Listing 8: Majority Voting Cluster Alignment and Confusion Matrix Extraction

```

1 from scipy.stats import mode
2
3 def align_clusters(labels, true_labels):
4     aligned = np.zeros_like(labels)
5     for c in np.unique(labels):
6         mask = (labels == c)
7         true_mode = mode(true_labels[mask], keepdims=True).mode[0]
8         aligned[mask] = true_mode
9     return aligned
10
11 kmeans_aligned = align_clusters(kmeans_labels, y)
12 hier_aligned = align_clusters(hier_labels, y)
13
14 cm_kmeans = confusion_matrix(y, kmeans_aligned)
15 cm_hier = confusion_matrix(y, hier_aligned)

```

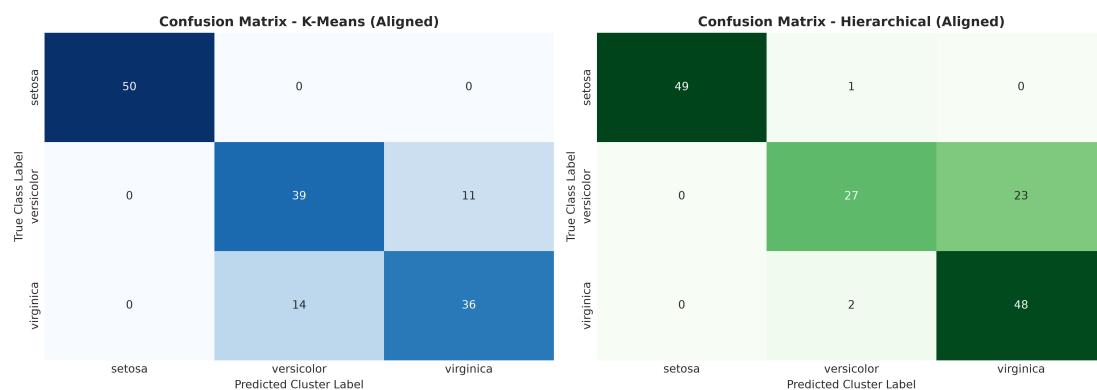


Figure 8: Confusion matrices for aligned K-Means (Left) and Hierarchical Clustering (Right) evaluated against ground-truth Iris species.

Contingency Breakdown:

- **K-Means:** Accurately clusters all 50 *Setosa* instances (100%). For *Versicolor*, 39 instances are classified correctly while 11 overlap into the *Virginica* cluster. For *Virginica*, 36 instances are classified correctly while 14 fall into the *Versicolor* cluster. Total accuracy: 83.33%.
- **Hierarchical (Ward):** Accurately clusters 49 *Setosa* instances (1 placed in *Versicolor*). For *Versicolor*, 27 are grouped into cluster 1, while 23 bridge into the *Virginica* cluster. For *Virginica*, 48 are correctly grouped, with only 2 misassigned. Total accuracy: 82.67%.

6 Comparative Discussion and Conclusions

6.1 Comparative Algorithmic Tradeoffs

Table 2: Algorithmic Comparison: K-Means vs. Hierarchical Agglomerative Clustering		
Property	K-Means Clustering	Hierarchical (Ward)
Computational Complexity	$O(N \cdot K \cdot d \cdot I)$ (Scalable to large datasets)	$O(N^2 \log N)$ time, $O(N^2)$ memory (Restricted to small/medium datasets)
Cluster Shape Assumption	Convex, isotropic, spherical clusters around centroids	Flexible based on linkage (Ward favors equal-variance spherical clusters)
Determinism	Stochastic (sensitive to centroid seed initialization)	Fully deterministic given distance matrix
Dendrogram Output	No hierarchical hierarchy produced	Produces complete, cuttable multiscale tree
Best Performance Metric	Higher Silhouette (0.4599) and Calinski-Harabasz (241.90)	Lower Davies-Bouldin Index (0.8035)

6.2 Analytical Insights

1. **Feature Scaling Necessity:** Preprocessing with *z*-score normalization is essential. Because Euclidean distance weights all dimensions equally, unscaled features with larger variance (such as Sepal Length) would dominate distance calculations and distort cluster topology.
2. **Boundary Ambiguity:** Both algorithms cleanly separate *Iris setosa* from the rest of the dataset due to substantial Euclidean margin in petal dimensions. However, the continuous morphometric gradient between *Iris versicolor* and *Iris virginica* leads to boundary crossover in both models, which aligns with biological taxonomy observations.
3. **Metric Agreement:** K-Means achieves a slightly higher Silhouette score (0.4599 vs. 0.4467) and Calinski-Harabasz score (241.90 vs. 222.72), while Hierarchical clustering achieves superior Davies-Bouldin compactness (0.8035 vs. 0.8336).

6.3 Conclusion

This experimental study successfully implemented, evaluated, and compared K-Means and Hierarchical Agglomerative clustering on the Iris dataset. Hyperparameter sweeps using the Elbow Method and Silhouette analysis confirmed $k = 3$ as the optimal natural partition count. Both algorithms achieved over 82% unsupervised alignment with ground truth, demonstrating the power of geometric distance-based partitioning for unsupervised feature discovery.